

▼ Partition-based clustering

Prerequisites

Students should know of:

- Clustering basics
- K-Means

Learning Objective

After reading this notebook, students should be able to:

- State the K-Means algorithm with formal intuition of the objective function.
- Identify why gradient-based optimization cannot be used in K-Means and explain the coordinate descent optimization approach.
- Examine the effect of centroid initialization in convergence and describe the various initialization methods.

▼ K-Means Algorithm

K-Means is an iterative algorithm that tries to separate a given dataset into K number of clusters and minimize the distance between data points and their respective cluster centroid. It is one of the simplest and popular clustering algorithms.

The whole process of K-Means is summarized in the following few steps:

Input:

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$$

where,

- N = total number of samples
- $\mathbf{x}_i = i^{th}$ sample from the dataset X where $\mathbf{x}_i \in \mathbb{R}^Z$
- Z = total number of features/attributes

Output: Vector $\mathbf{c}_N$ of cluster assignments in $\mathbb{R}^N$, and K number of mean vectors $\boldsymbol{\mu}_k$ where matrix of all K means is in $\mathbb{R}^{K \times Z}$

Steps:

1. Specify the number of clusters K .
2. Initialize the cluster centroids with any initialization methods.
3. Compute the distance between each point and each cluster centroids (Euclidean distance metric is a popular choice) Where distance is a vector in $\mathbb{R}^N$ for all data points and for all

K centroids, distances will be a matrix in $\mathbb{R}^{K \times N}$.

$$distance = \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$$

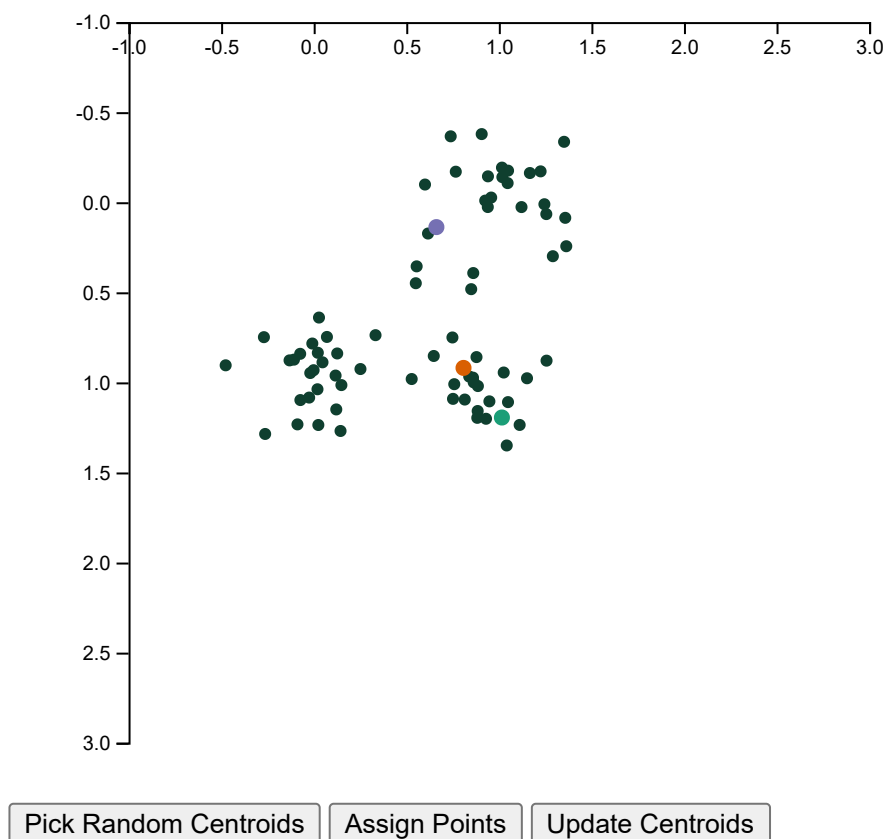
4. Assign each point to the closest cluster centroid (minimum distance).
5. Compute the new cluster centroid for each cluster by taking the mean of all the data points in that cluster.
6. Repeat steps 3-5 until there is no change in the cluster centroids.

Note: Initialization (Step 2) can be done by randomly sampling the centroids from the range of the data or by using methods like K-Means++. We will discuss more on the initialization later in this chapter.

We can also see and test the above defined few steps in the interactive widget below:

Experimental Cell: KMeans Clustering

[Show code](#)



In the widget above, we can visually see how K -Means clustering works and interact with the convergence process.

Here we have pre-defined dataset and 3 cluster centroids. To randomly pick the centroids from the data points, you can press the `Pick Random Centroids` button. After that you can assign the datapoints to the cluster of closest centroid by pressing `Assign Points` button.

Now that the data points are assigned, you can further update the centroids by pressing **Update Centroids** button to calculate mean value and update to new centroid. You can repeat this process of assigning and updating until there is no change in the cluster centroids.

Note: To reset the points assignments, you can reload the page or if you want to pick random centroids again, you can do it by pressing the same **Pick Random Centroids** button.

▼ K-Means Objective Function

The K -Means algorithm's objective is to minimize the sum of squared distance between data points and the centroid within a cluster and maximize the distance between different clusters.

We can use the following objective function to minimize the distance between data points and the centroid which makes a tighter cluster and distance between other clusters are automatically maximized.

$$\mathcal{J} = \arg \min_{\mu, c} \sum_{k=1}^K \sum_{i=1}^N 1\{c_i = k\} \|\mathbf{x}_i - \mu_k\|_2^2 \quad (1)$$

Where,

- $\mathcal{J}$ is the objective function.
- K is the number of clusters and N is the total number of samples.
- $\mathbf{x}_i$ is the vector of i^{th} data point.
- c is a cluster assignments vector which contains the index of k^{th} cluster in which i^{th} data point belongs to.
- $1\{c_i = k\}$ is an indicator function of a set which equals to 1 if the $c_i = k$ else 0.
- μ_k is the vector of k^{th} cluster centroids:

$$\mu_k = \frac{\sum_{i:c_i=k}^N \mathbf{x}_i}{|c_{i=k}|}. \quad (2)$$

In a plain language, above objective function in equation (1) finds the μ and c which minimizes the distance between $\mathbf{x}_i$ and μ_k where the given $\mathbf{x}_i$ is in cluster assignments $c_{i=k}$.

Note: The objective function of K-Means is referred to as Sum of Squared Error (SSE) or the Residual Squared Error (RSS).

The above-given equation(1) is the formal objective function of K -Means, and equation(2) shows how we can update the k^{th} cluster centroids by taking mean among the data points on that cluster. It is hard to comprehend so, we will break it down later, but first, let us know something about the algorithm.

Since K -Means is an **iterative algorithm**, its objective function cannot be optimized by taking derivatives and setting zero. **We have to use some iterative optimization algorithms like Gradient Descent, but the problem here is our objective function is composed of two dependent unknowns: μ and c . Gradient Descent algorithm attempts to update all parameters at same time but we cannot find their best values at the same time to minimize our objective function $\mathcal{J}$.** However, we can fix the value of μ and find the best c , and after that, we can fix the value of c and find the best μ .

This process of holding on one set of parameters fixed and optimizing the other, and vice-versa is called the **Coordinate Descent** optimization approach.

▼ Coordinate Descent

Let's see our full objective function from equation(1) again:

$$\mathcal{J} = \arg \min_{\mu, c} \sum_{k=1}^K \sum_{i=1}^N 1\{c_i = k\} \|\mathbf{x}_i - \mu_k\|_2^2$$

As discussed above K -Means iterate between two steps. So, we can break our objective function as following:

1. Assigning observations to the nearest cluster prototype:

$$c_i \leftarrow \arg \min_k \|\mathbf{x}_i - \mu_k\|_2^2 \quad (3)$$

Here c is a vector which contains the assignment indexes of the k^{th} centroid in which i^{th} data point belongs to.

2. Updating the cluster centers:

$$\mu_k = \frac{\sum_{i:c_i=k}^N \mathbf{x}_i}{|c_{i=k}|}$$

Now, after assigning data to the nearest cluster for previous centroid μ_k , we have to update new centroid μ_k by taking mean of all the i^{th} datapoints in cluster prototype c where $c_{i=k}$. Since computing the mean of a set of observations is equivalent to computing its center of mass, We can re-write our second equation as a minimization objective:

$$\mu_k \leftarrow \arg \min_{\mu} \sum_{i:c_i=k}^N \|\mathbf{x}_i - \mu\|_2^2. \quad (4)$$

From the above equation 3 and 4, we can see that we are iterating between two steps where each step represents minimization of some objective. Now we can formalize that this is an example of **Coordinate Descent optimization**.

Let's look at the short python snippet for the implementation of coordinate descent optimization.

```

centers = <array of centroids>
data = <array of input features>
n_centers = <number of centroids(k)>
dis_array = []
threshold = 0.001 # Threshold to check optimal

## Calculate euclidean distance
-----
for center in centers:
    dis = np.sqrt(np.sum(data - center)**2)
    dis_array.append(dis)

## Assign cluster to the nearest center
-----
data_cluster = np.argmin(dis_array, axis=0) # getting index of minimum over the the centers

## Update centroids
-----
## make copy of previous centers and update the centers
prev_centers = np.copy(centers)
for i in range(n_centers):
    centers[i] = np.mean(data[data_cluster == i], axis=0) # mean among the row

## Check if optimal
is_optimal = False

non_optimal = np.abs(prev_centers - centers) > threshold
if non_optimal.astype(int).sum() == 0:
    is_optimal = True
is_optimal = False

```

Convergence of K -Means

Now that we know how we can optimize our objective function $\mathcal{J}$ let's see about the convergence of this algorithm.

In K -Means, given a particular μ , we may be able to find the best c , but once we update c , we can probably find a better μ and vice-versa. This updating seems like a forever oscillating iterative process since μ and c depend on each other, but it is guaranteed that K -Means will converge in finite steps. This convergence can be proven by pointing out that the squared distance between the instances and the closest centroid can only go down at each step.

Note: You can refer to the Exercise 9.1 of PRML book by Bishop to see the proof.

The objective function of $\mathcal{J}$ monotonically decreases with each iteration. Every update to $\mathbf{c}$ or $\boldsymbol{\mu}$ decreases $\mathcal{J}$ compared to the previous value. Although the algorithm is guaranteed to converge, it may not converge to the right solution, i.e., it may converge to the local optimum. This local optimum is a result of $\mathcal{J}$ being non-convex.

Non-convexity means that different initializations will give different results. Let's look at the example:

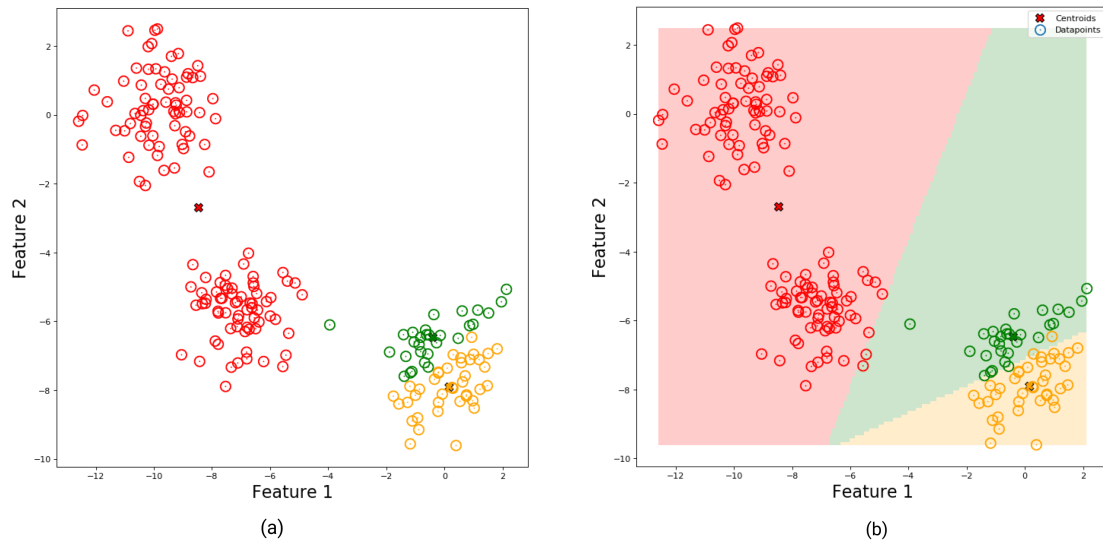


Figure 2: (a) Cluster centroids random initialization,
(b) Clustering decision region

In the figure above, while randomly sampling and initializing the centroid from the dataset (Forgy initialization method), two centroids have been initiated in the bottom right cluster, and one centroid has been initiated in the top. Now, while converging, one big red cluster is formed top, and in the bottom right, we have two small clusters. This is the result of poor centroid initialization.

K-Means is very sensitive to the initialization. As we have seen above by converging on local optimum, we are getting poor results. To avoid this poor performance, the algorithm can be run multiple times with different initializations. The results with the lowest $\mathcal{J}$ can be picked, but other initialization methods can also help in the clustering performance.

Centroid Initialization

There are various methods of initializing the initial cluster centroids. Earlier while defining steps of K-Means, we ignored the initialization step, so here we are going to look at some of the popular initialization methods for continuous features to improve the performance.

Forgy Initialization

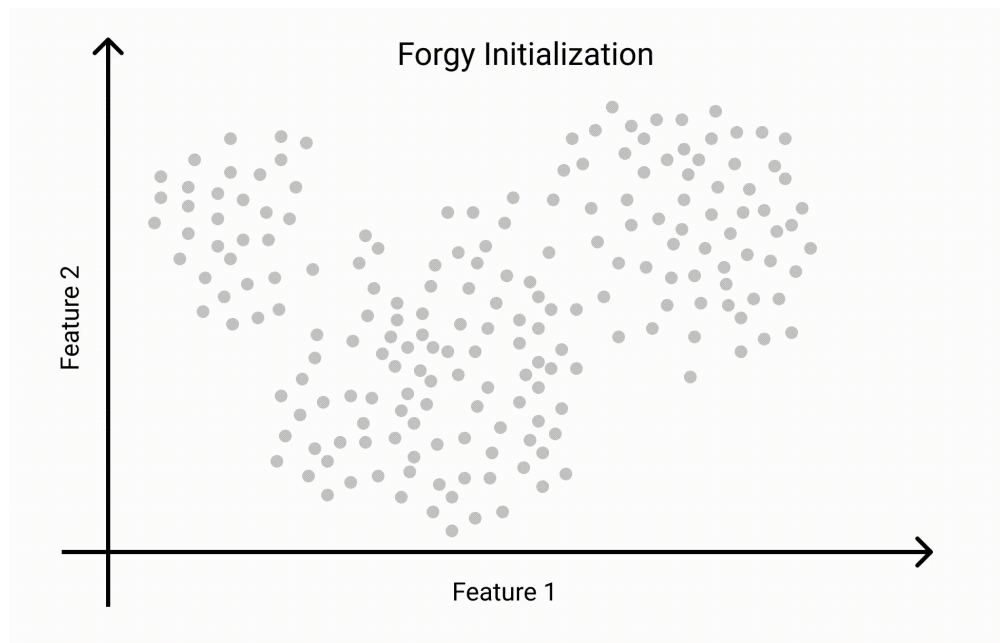


Figure 3: Forgy initialization

Forgy initialization is the simplest method of cluster centroid initialization. In Forgy initialization, we pick the random K points from the available data points and assign them as a cluster centroids. As we have seen above, there is a problem with randomly picking data point as a centroid. If we are lucky, then our clusters might converge properly, but it is not always the same.

```
def forgy_initialization(data, k):
    """Forgy initialization

    Parameters:
        - X: Data
        - k: number of centers
    """

    rand_index = np.random.randint(
        low=0, high=data.shape[0], size=k
    )
    centers = data[rand_index]
    return centers
```

K-Means++

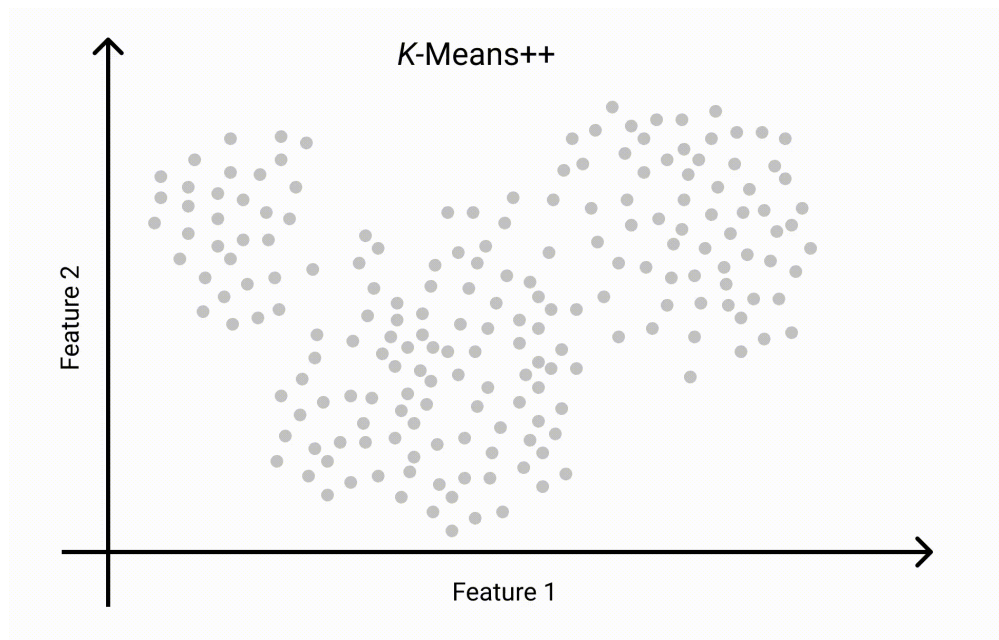


Figure 4: K-Means++ initialization

The *K-Means++* is the popular centroid initialization method. This algorithm carefully selects the initial centroids for *K-Means* clustering. It follows a simple probability-based approach where the first centroid is selected at random and after that, the squared distance is calculated for all the data points with that centroid. For the next centroid, a data point that has the largest distance has the highest probability of being selected. This selection is decided based on a weighted probability score which we will also see in the code snippet below. The selection is continued until we have *K* centroids, and then *K-Means* clustering is done using these centroids. Let's look at the steps:

Algorithm KMeans++

1. Initialize *K* and pick first initial centroid at random from the dataset.
2. Calculate distance from the first centroid to all data points.
3. Pick next centroid which is farthest from the first centroid using some weighted probability score.
4. Repeat 2-3 until *K* clusters reached.

As you can also see in the animation above, *K-Means++* is selecting the farthest points as centroids which leads to the faster convergece. It is better than the forgy initialization, but it is computationally expensive as we are calculating distances with every datapoint for each centroid.

```
def kmeans_plus_plus_initialization(X, k):
    """KMeans++ initialization
```

Parameters:

- X: Data
- k: number of centers


```
"""
```

```
centers = []
X = np.array(X)

## Sample the first point at random
initial_index = np.random.choice(range(X.shape[0]),)
centers.append(np.asarray(X[initial_index, :].tolist()))

## Loop and select the remaining points
for i in range(k - 1):
    distance = np.sum((np.array(centers) - X[:, None, :])**2, axis = 2)

    if i == 0:
        pdf = distance / np.sum(distance)
        centroid_new = X[
            np.random.choice(range(X.shape[0]), replace=False, p=pdf.flatten())
        ]
    else:
        ## Calculate distance of each point from its nearest centroid
        dist_min = np.min(distance, axis=1)

        pdf = dist_min / np.sum(dist_min)

        ## Sample one point from the given prob distribution
        centroid_new = X[np.random.choice(range(X.shape[0]), replace=False, p=pdf)]

    centers.append(centroid_new.tolist())

return np.array(centers)
```

There have been various research on the centroid initialization, and different other methods have been developed. Similarly, for categorical features, multiple methods like *Cao, Huang* are available. We are not going to learn about those methods in this chapter because we can use forgy initialization for both types, and it works well.

Note: You can refer to the *Experimental Analysis* section of the paper *Cluster Center Initialization Algorithm for K-modes Clustering* by Shehroz S. Khan (2013) to learn about variant of KMeans for categorical features.

Number of clusters and distance metric

Another factor that affects *K*-Means clustering is the choice of *K* value and the distance metric.

K in K -Means is the hyperparameter, which is to be initialized at the first step of K -Means. As discussed earlier, it defines how many clusters we want to create. Earlier, we just mentioned to initialize the value for K , but we don't know how to pick a value for K .

The K Means algorithm in its simplest form does not optimize the number K . The obvious next step from the K Means algorithm would be to optimize K using SSE, but this approach has a major flaw. The greater the number you use for K , the smaller value of the Sum of Squared Error (SSE) you get.

In fact, using $K = N$, where N is the number of points in our dataset, can give us an SSE of 0. It is impossible to do better than this, but it doesn't make sense to assign each point to its own unique cluster. Therefore, we need to do better than just minimize the SSE using K .

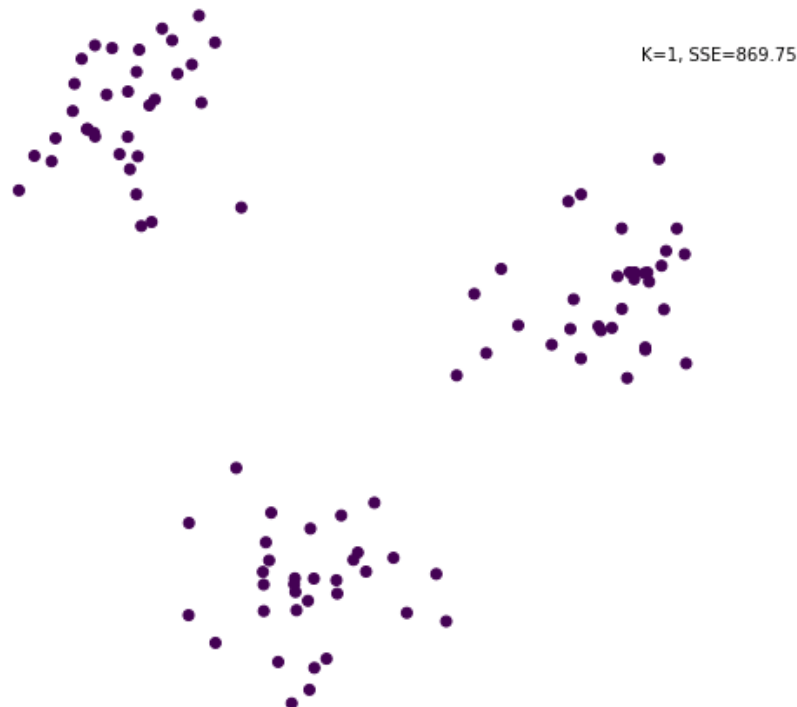


Figure 5: Larger values of K never give larger values of SSE

Previously in the K -Means chapters, we saw how we could pick the value for K using the **Elbow method**. Though we used the Elbow method, it is not very efficient, and it is not guaranteed that elbow is always formed. So, like other hyperparameters in Machine Learning problems, it is a challenging task to pick the right value for K , and domain knowledge also helps to choose the value for K . In the upcoming chapters we will learn about the **Silhouette coefficient** and other methods to measure the goodness of the cluster. There has also been research in several

methods like *Gap Statistic*, *Akaike Information Criterion(AIC)*, *Bayesian Information criterion(BIC)*, etc. but it is out of the scope for this unit.

Note: Please refer to the Chapter 4 of *Data Clustering: Algorithms and Applications Book by Charu C. Aggarwal* to learn about AIC and BIC

Distance metric is another hyperparameter which we have to choose on our own. *K-Means* has been designed to use the euclidean distance function but we can tweak this and use absolute distance to make the algorithm resilient to outliers or we can even use median instead of mean values to make it more robust. In such way we can build different variants on top of *K-Means* to patch over different limitations of *K-Means*. We will look at the variants in next chapter but for now let's look at some of the advantages and limitations of *K-Means*.

Advantages and Limitations of *K-Means*

K-Means algorithm is popular but it has some limitations as well, some of them are mentioned below:

Advantages of *K-Means*

- *K-Means* is the easiest clustering algorithm used widely in the industries.
- *K-Means* has the flexible framework due to which different variants can be built on top of the algorithm.

Limitations of *K-Means*

- *K-Means* works for numerical features only and choosing the value for *K* is challenging.
- Due to the Mean and squared distance based framework of *K-Means*, they make the algorithm highly sensitive to the outliers.
- *K-Means* algorithm takes the whole dataset as an input so it is not computationally efficient for the large datasets.
- *K-Means* algorithm assumes that data have some spherical shapes. Hence, they work poorly for non-convex data and cannot distinguish the outliers.

Key Takeaways

- *K-Means* is an iterative algorithm.
- We use Coordinate Descent optimization approach to optimize the objective function of *K-Means*
- *K-Means* is very sensitive to the cluster centroid initialization.

- K-Means++ is the popular centroid initialization method that uses a weighted probability

References

- Books
 - Bishop C. M. (2006), Pattern recognition and machine learning,
 - Exercise 9.1: convergence of K-Means is assured.
 - Charu C. Aggarwal. (2014), Data Clustering: Algorithms and Applications.
 - Chapter 4: A Survey of Partitional and Hierarchical Clustering Algorithms
 - Rui Xu. (2009), Clustering
 - Chapter 4: PARTITIONAL CLUSTERING
- Papers
 - David Arthur (2006), [K-Means++: The advantages of a careful seeding](#)
 - 2.2: The k-means++ algorithm
 - Shehroz S. Khan (2013), [Cluster Center Initialization Algorithm for K-modes Clustering](#)
 - 5: Experimental Analysis
- Articles
 - Scikit-learn, [Plot KMeans Silhouette Analysis](#)
 - Silhouette Analysis